Vietnam National University - HCMC University of Science

Faculty of Information Technology

---□🕮□---

Requirement Analysis

Course: Introduction to Software Engineering

Students that participated in this project:

Đinh Ngọc Anh Dương – 23127039

Trần Thị Xuân Tài – 23127256

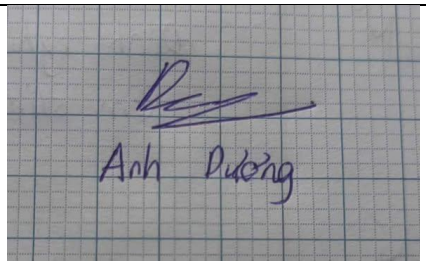
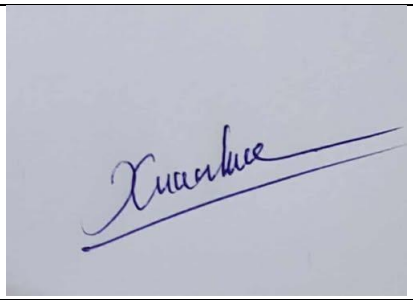
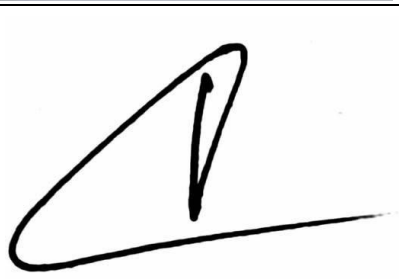
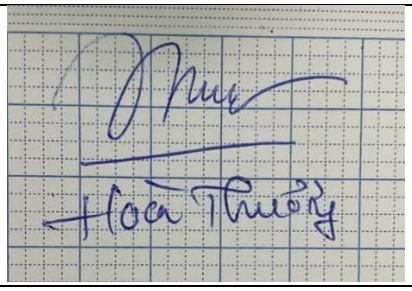
Trần Thọ - 23127264

Nguyễn Tuấn An – 23127316

Đỗ Thị Hoài Thương - 23127492

Instructor: Trương Phước Lộc

I. Member Contribution Assessment

ID	Name	Contribution(%)	Signature
23127039	Đinh Ngọc Anh Dương	100%	
23127256	Trần Thị Xuân Tài	100%	
23127264	Trần Thọ	100%	
23127316	Nguyễn Tuấn An	100%	
23127492	Đỗ Thị Hoài Thương	100%	

II. Problem Statement

1. Introduction

The Programming Practice System is an online platform that helps learners improve their programming and problem-solving skills. It provides a collection of programming problems, a built-in web IDE, automated judging, and progress tracking. The system supports students, self-learners, and developers preparing for technical interviews or strengthening basic programming skills.

2. Business Context and Motivation

Traditional programming education lacks automation, feedback, and structured problem access. Learners often struggle to measure progress or practice effectively. This system addresses these issues by offering:

- A searchable problem library
- A browser-based IDE
- Automated assessments with instant feedback
- Submission management and drafts
- Progress analytics
- Secure authentication and user profiles

3. Software Problem Description

3.1 User Authentication and Profile

Users can register, log in, and update personal information. Password handling follows strict security standards, and sessions remain protected at all times.

3.2 Problem Library

The system organizes problems by difficulty, topic, and tags. Each problem includes descriptions, constraints, examples, and test cases. Filters and search tools help users find suitable challenges.

3.3 Online Code Editor

A web-based editor supports languages such as Python, Java, C++, and JavaScript. Features include syntax highlighting, auto-completion, indentation, and error hints. Users may write code directly or load previous drafts.

3.4 Code Submission and Review

Submitted code is compiled and executed inside a secure environment. The judge returns results such as correctness, errors, runtime exceptions, or timeouts, along with performance metrics.

3.5 Submission History & Analytics

Users can view all past submissions, compare attempts, and track performance. Analytics like solved count, accuracy, and progress trends help identify strengths and weaknesses.

3.6 Problem Management

Administrators can create, edit, update, or archive problems. Test cases and metadata can be adjusted as needed, and all admin actions are logged.

4. Operating Environment

Client: Modern browsers, Monaco-based editor
Server: React.js frontend, Spring Boot backend, Redis queue
Judge: Docker sandbox, multi-language runtime, resource limits
Database: PostgreSQL for structured data, MongoDB for logs
Storage: Google Drive, documentation via GitHub Pages/ReadTheDocs

5. Design & Implementation Constraints

• Backend in Java; frontend in React
• Multi-language editor with sandboxed execution
• SSL/TLS encryption, bcrypt/Argon2 hashing, anti-brute-force protection
• Fast performance: page loads <10s, judge <30s
• High reliability and horizontal scalability

III. Requirements Overview

1. Stakeholders

Stakeholder Table

No.	Stakeholder	Description / Role
1	**Course Instructor**	Acts as the project initiator and requirement provider. Defines the system goals, monitors the progress, and evaluates the final results. Functions similarly to a "customer" in a real software development project.
2	**End User**	Represents both new users and registered users. They interact directly with the system to: register an account, log in, browse problems, write and run code, submit solutions, view results, and track submission history. They are the primary audience of the application.
3	**Administrator**	Manages the system's content: creating, updating, and deleting problems. Responsible for monitoring user statistics, submission activities, and maintaining overall system integrity.
4	**Judging System**	The automated component that compiles, executes, and evaluates user-submitted code. Provides accurate results including correctness, runtime, and errors. Must respond within ≤10 seconds to ensure good user experience.
5	**System Infrastructure**	Includes servers, databases, and the secure code execution environment. Ensures performance, security, and high system availability with monthly downtime <1%.
6	**Development Team**	The student team responsible for analyzing requirements, designing the system, implementing features, testing functionality, and delivering the project according to the instructor's expectations.

2. Requirements

2.1 Functional Requirements Specification

The system shall provide the following functional capabilities:

1. User Account Management

1.1. The system shall allow new users to register an account by providing required information.
1.2. The system shall authenticate users through a login process using a valid username and password.
1.3. The system shall store user passwords in a secure hashed format.

2. Problem Browsing and Filtering

2.1. The system shall display a list of programming problems.
2.2. The system shall categorize problems by difficulty levels (Easy, Medium, Hard).
2.3. The system shall allow users to filter problems by difficulty.

3. Online Code Editor and Code Execution

3.1. The system shall provide an online code editor for users to write and modify source code.
3.2. The system shall allow users to submit their code for execution.
3.3. The system shall compile and run the submitted code in an isolated execution environment.
3.4. The system shall return the execution results (correct/incorrect, runtime, errors) to the user.

4. Automatic Judging System

4.1. The system shall compare user submissions with predefined test cases.
4.2. The system shall evaluate correctness, performance, and runtime errors.
4.3. The system shall provide feedback within a maximum response time of **10 seconds** per submission.

5. Submission History and Progress Tracking

5.1. The system shall store all user submissions.
5.2. The system shall allow users to view past submission attempts and their results.
5.3. The system shall display statistics such as accepted submissions and failed attempts.

6. Administrative Functions

6.1. The system shall allow administrators to create, update, and delete problems.
6.2. The system shall allow administrators to manage test cases associated with each problem.
6.3. The system shall provide administrators with access to system usage analytics and submission statistics.

2.2 Non-Functional Requirements Specification

The system must satisfy the following non-functional requirements:

1. Performance Requirements

1.1. The system shall process code submissions and return results within 10 seconds.
1.2. The system shall support concurrent execution of multiple submissions without significant performance degradation.

2. Reliability and Availability

2.1. The system shall maintain high operational stability.
2.2. The system's availability shall be at least 99% per month (maximum downtime < 1% per month).
2.3. The judging service shall recover automatically from runtime failures.

3. Security Requirements

3.1. The system shall store all user passwords using secure hashing algorithms.
3.2. The system shall sanitize and validate user inputs to prevent SQL injection and XSS attacks.
3.3. The code execution environment shall be sandboxed to prevent unauthorized access to the server.

4. Usability Requirements

4.1. The user interface shall be intuitive and easy to navigate.
4.2. The system shall be accessible to new users without requiring prior platform knowledge.
4.3. The system shall support clear display of code execution results and error messages.

5. Scalability Requirements

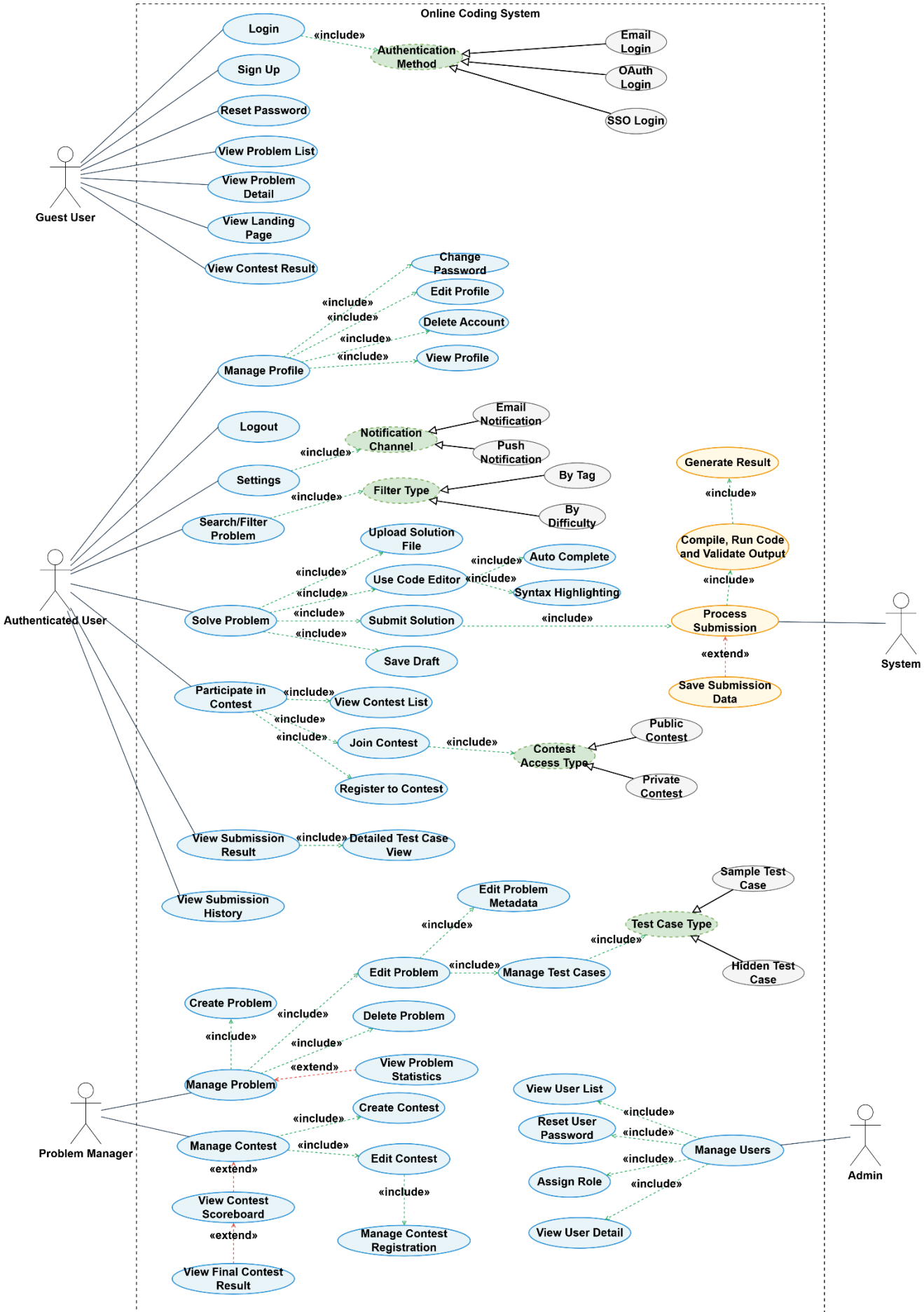
5.1. The system shall be able to scale to support an increasing number of users and submissions.
5.2. The system architecture shall allow horizontal scaling of judging servers.

6. Maintainability Requirements

6.1. The system codebase shall follow clean coding practices to ensure ease of maintenance.
6.2. The system shall support modular design to simplify updates and feature extension.

IV. Requirements Analysis

1. Use Case model


Online Coding System
Guest User
Login
«include»
Authentication Method
Email Login
OAuth Login
SSO Login
Sign Up
Reset Password
View Problem List
View Problem Detail
View Landing Page
View Contest Result
Authenticated User
Manage Profile
Change Password
Edit Profile
Delete Account
View Profile
Logout
Settings
Notification Channel
Email Notification
Push Notification
Search/Filter Problem
Filter Type
By Tag
By Difficulty
Solve Problem
Upload Solution File
Use Code Editor
Auto Complete
Syntax Highlighting
Submit Solution
Save Draft
Generate Result
Compile, Run Code and Validate Output
Process Submission
Save Submission Data
System
Participate in Contest
View Contest List
Join Contest
Register to Contest
Contest Access Type
Public Contest
Private Contest
View Submission Result
Detailed Test Case View
View Submission History
Problem Manager
Manage Problem
Create Problem
Edit Problem
Delete Problem
View Problem Statistics
Edit Problem Metadata
Manage Test Cases
Test Case Type
Sample Test Case
Hidden Test Case
Manage Contest
Create Contest
Edit Contest
View Contest Scoreboard
View Final Contest Result
Manage Contest Registration
Admin
Manage Users
View User List
Reset User Password
Assign Role
View User Detail
«include»
«extend»

2. Use Case Specification

2.1. UC01 – User Login

Use case ID	UC01
Use Case	User Login
Brief Description	User authenticates with email/username and password to access personal features of the system.
Actor	Guest (future Student/User, Problem Setter, Admin)
Pre-Condition	User has a registered account. User is on the Login page. System is online.
Result	On success: user is logged in and redirected to home/dashboard. On failure: user remains on Login page with error message.
Main Scenario	1. User opens Login page. 2. System displays login form (email/username, password). 3. User enters credentials and clicks Login. 4. System validates input format. 5. System checks credentials against user database. 6. System creates authenticated session/JWT and loads user role & settings. 7. System redirects user to home/dashboard (with personalized data).
Alternative Scenarios	A1 – Invalid credentials: at (5) no match → system shows "Invalid username or password" → back to (3). A2 – Account locked/inactive: at (5) account flagged → system shows "Account locked, contact admin" → login denied. A3 – System/DB error: at (5) database not reachable → system logs error and shows generic "Login service unavailable".
Non-Functional Constraints	Response time < 2 seconds under normal load. Credentials must be transmitted via HTTPS. Passwords stored with secure hashing (e.g., bcrypt). Brute-force protection (lock/throttle after N failed attempts).

2.2. User Sign Up

Use case ID	UC02
Use Case	User Sign Up
Brief Description	Guest creates a new account to use practice and contest features.
Actor	Guest
Pre-Condition	User is not logged in. Registration is enabled by the system.
Result	On success: new user account created; user can log in.
Main Scenario	1. Guest opens Sign Up page. 2. System shows registration form (username, email, password, confirm password). 3. Guest fills in required fields and clicks Sign Up. 4. System validates input (format, required fields, password rules). 5. System checks email/username uniqueness. 6. System creates new user with default role User and default settings. 7. System shows success message and provides link to Login page.
Alternative Scenarios	B1 – Invalid data: at (4) invalid format → system highlights fields and shows errors → back to (3). B2 – Email/username already used: at (5) conflict → system shows "Email/username already exists" → back to (3). B3 – System error: at (6) DB fails → system logs error and shows generic failure message.
Non-Functional Constraints	Registration must complete in < 3 seconds. Password must meet minimal complexity (length, character rules). Registration endpoint rate-limited to avoid spam.

2.3. View Problem List

Use case ID	UC03
Use Case	View Problem List
Brief Description	User browses list of available problems with optional filter/search.
Actor	Guest, Authenticated User
Pre-Condition	System has at least one problem defined. User is on Problems page or navigates there from landing/home page.
Result	System displays problem list; user may apply filters/search.
Main Scenario	1. User opens Problems page. 2. System retrieves list of visible problems from database. 3. For authenticated users, system determines status (Solved/Attempted/Unsolved) per problem. 4. System displays table/grid (title, difficulty, tags, status).
Alternative Scenarios	C1 – Apply filter/search: after (4) user selects tag/difficulty/status or enters keywords → system re-queries and updates list. C2 – No problems match filter: query returns empty set → system shows "No problems found" and keeps filter visible.
Non-Functional Constraints	First page of results must load in < 2 seconds for up to N problems. Pagination or infinite scroll required for large problem sets.

2.4 View Problem Detail

Use case ID	UC04
Use Case	View Problem Detail
Brief Description	User views full statement and metadata for a selected problem.
Actor	Guest, Authenticated User
Pre-Condition	UC03 or direct link provides a valid problem ID. Problem is visible to the user (not hidden/private).
Result	Problem details (statement, examples, limits) shown; editor and submission options available if allowed.
Main Scenario	1. User clicks a problem from list (or opens by direct URL). 2. System loads problem by ID from database. 3. System displays: title, statement, input/output format, constraints, samples, tags, difficulty, time/memory limits.

	4. If user is authenticated, system also shows Solve in editor, Upload solution, View submissions actions.
Alternative Scenarios	D1 – Problem not found: at (2) record missing → system shows "Problem not found". D2 – Unauthorized (private/contest problem): at (2) access not allowed → system shows access denied message.
Non-Functional Constraints	Page should render in < 2 seconds. Support math/code formatting and mobile-friendly layout.

2.5 Solve Problem in Integrated Editor & Save Draft

Use case ID	UC05
Use Case	Solve Problem in Integrated Editor & Save Draft
Brief Description	Authenticated user writes solution in built-in editor, optionally uploads a file, and saves code as draft without grading.
Actor	Authenticated User
Pre-Condition	UC04 (View Problem Detail) completed. User is logged in.
Result	Draft solution is stored and can be reloaded later; no grading performed.
Main Scenario	1. From problem detail, user opens Editor. 2. System displays code editor with language selector and current draft (if exists). 3. User selects language (e.g., C++, Java, Python). 4. User writes/pastes code in editor. 5. Optionally, user uploads code file → system loads file content into editor. 6. User clicks Save Draft. 7. System validates basic input (language chosen, code not empty). 8. System saves/updates draft record linked to user + problem + language. 9. System confirms "Draft saved" and keeps user on editor page.
Alternative Scenarios	E1 – Empty code: at (7) code empty → system shows "Code cannot be empty" → back to (4–5). E2 – Session expired: at (6–8) session invalid → system redirects to Login; after login, user may be redirected back to

	editor.
Non-Functional Constraints	Draft save must complete in < 1 second. Draft storage size per user/problem limited (e.g., max file size). Editor auto-save (optional) must not overload server.

2.6. Submit Solution (Practice)

Use case ID	UC06
Use Case	Submit Solution (Practice)
Brief Description	User submits solution (from editor or file upload) for grading in practice mode.
Actor	Authenticated User, Judge System (secondary actor)
Pre-Condition	User logged in. Valid problem visible to user. User has written or uploaded code.
Result	Submission is stored, judged, and verdict/log are available to user.
Main Scenario	1. User is on problem editor page with code ready. 2. User selects language (or confirms default). 3. User clicks Submit. 4. System validates input (language supported, code not empty). 5. System creates submission record with status = "Queued". 6. System sends submission to Judge System (e.g., via message queue). 7. Judge System compiles and runs code against all test cases in sandbox. 8. Judge System determines verdict (AC/WA/TLE/MLE/RE/CE) and metrics (time, memory). 9. Judge System sends result back to main system. 10. System updates submission record with final verdict and metrics. 11. System notifies user (in UI) and shows verdict in submission panel.
Alternative Scenarios	F1 – Compile error: at (7) compilation fails → verdict = CE + compiler log → system stores and displays CE + log. F2 – Runtime/time/memory error: at (7) runtime exception or limit exceeded → verdict = RE/TLE/MLE → system stores and displays verdict.

	F3 – Judge unavailable: at (6) send fails or timeout → system marks submission as "Error/Pending" and shows message "Submission could not be judged, try again later." F4 – Invalid/empty code: at (4) fail → system shows validation message and does not create submission.
Non-Functional Constraints	Step (1–5) must respond in < 2 seconds; grading (7–10) may take longer but should normally finish < 30 seconds. Each execution must be fully sandboxed with CPU, memory, and time limits. System must handle at least N concurrent submissions without losing data.

2.7 View Submission Result & History

Use case ID	UC07
Use Case	View Submission Result & History
Brief Description	User reviews verdict and history of their submissions for a problem or overall.
Actor	Authenticated User
Pre-Condition	User logged in. User has at least one submission OR system shows empty state.
Result	System shows list of submissions and details for selected one.
Main Scenario	1. User opens My Submissions page or Submissions tab for a problem. 2. System retrieves user submissions (optionally filtered by problem/contest). 3. System shows table (time, problem, language, verdict, runtime, memory, source view link). 4. User selects a submission. 5. System displays submission detail (source code, verdict, test-case log).
Alternative Scenarios	G1 – No submissions: at (2) none found → system shows "You have no submissions yet". G2 – Accessing others' submissions: user tries to open submission not owned and not shared → system denies access.
Non-Functional Constraints	History list must support paging for large numbers of submissions. Source code view must be read-only and protected from unauthorized access.

2.8 Manage Account & Profile

Use case ID	UC08
Use Case	Manage Account & Profile
Brief Description	Authenticated user views and updates profile info, settings, and password; may delete account.
Actor	Authenticated User
Pre-Condition	User logged in.
Result	Profile/settings/password updated or account deleted.
Main Scenario	1. User opens Profile/Settings page. 2. System displays current profile (name, avatar, contact), notification/theme settings. 3. User edits fields (e.g., display name, avatar, theme) and clicks Save. 4. System validates input and saves changes. 5. System confirms “Profile updated”.
Alternative Scenarios	H1 – Change password: 1. From settings, user chooses Change Password. 2. System asks for current password and new password. 3. System validates current password and rules for new password. 4. If valid, system updates password and shows success. H2 – Delete account: 1. User clicks Delete account. 2. System asks for confirmation and possibly re-authentication. 3. On confirmation, system flags account as deleted/inactive and logs user out.
Non-Functional Constraints	Sensitive actions (change password, delete account) must require re-authentication or strong confirmation. All changes must be audited.

2.9 Manage Problem (Create/Edit/Delete/Metadata)

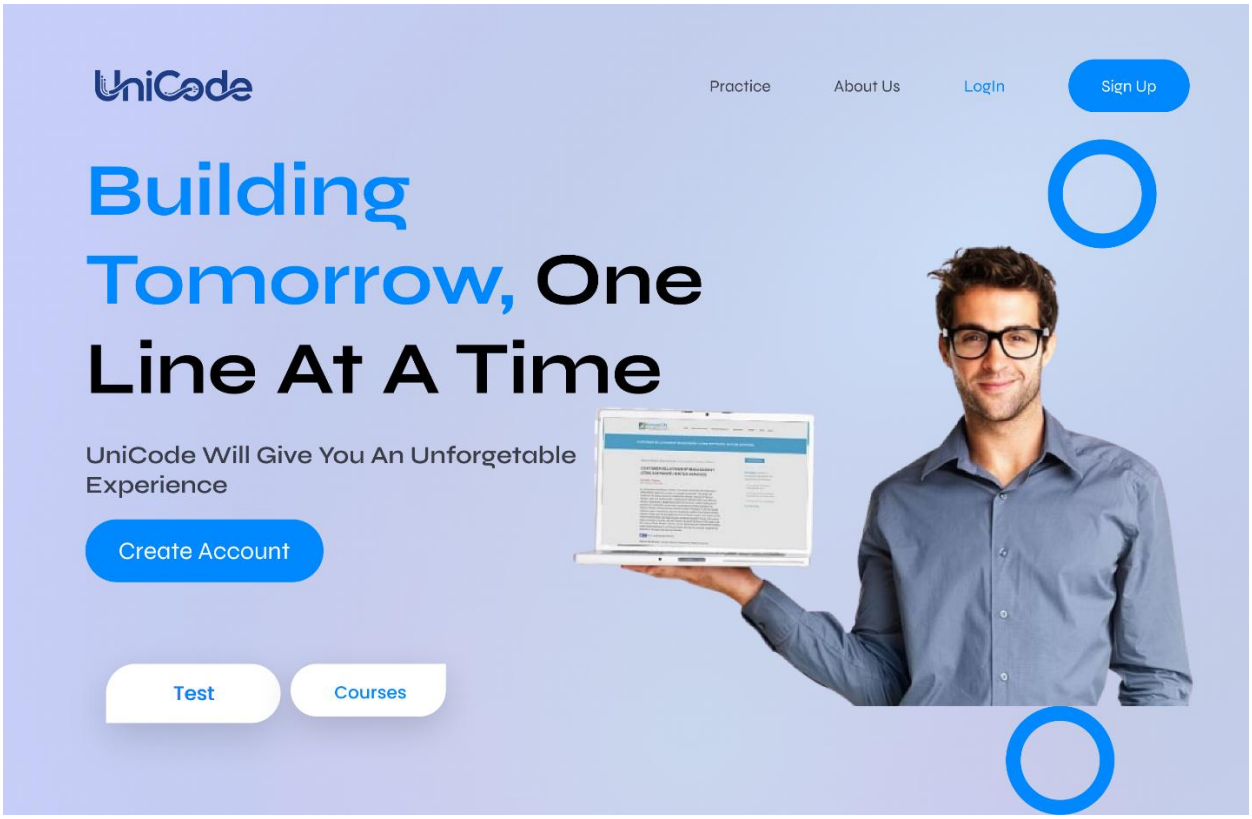
Use case ID	UC12
Use Case	Manage Problem (Create/Edit/Delete/Metadata)
Brief Description	Problem setter creates and maintains problems, including metadata and statistics.
Actor	Problem Setter / Problem Manager / Contest Organizer, Admin
Pre-Condition	Actor logged in with problem-management permission.
Result	Problem bank updated (new problem, modified problem, or deleted/archived problem).
Main Scenario	1. Actor opens Problem Management → Create Problem. 2. System shows problem creation form. 3. Actor fills statement fields (title, description, I/O description, constraints, notes). 4. Actor sets metadata (difficulty, tags, time/memory limits, visibility). 5. Actor adds sample and hidden test cases. 6. Actor clicks Save/Publish. 7. System validates data. 8. System saves problem and test cases. 9. System confirms success.
Alternative Scenarios	L1 – Edit problem: - Actor selects an existing problem → system loads current data. - Actor modifies statement/metadata/test cases and saves → system validates and updates record. L2 – Delete/Archive problem: - Actor selects problem → chooses Delete/Archive. - System checks constraints (e.g., used in past contests) and either marks as archived or prevents deletion with explanation. L3 – View problem statistics: - From problem management, actor opens Statistics. - System shows AC rate, number of submissions, per-verdict counts.
Non-Functional Constraints	Only authorized roles can modify or delete problems. All changes must be logged (who, when, what).

2.10. Manage Users (View, Assign Role, Reset Password)

Use case ID	UC14
Use Case	Manage Users (View, Assign Role, Reset Password)
Brief Description	Admin reviews users, updates roles, and resets passwords when necessary.
Actor	Admin
Pre-Condition	Admin logged in with admin permission.
Result	User roles/status/passwords updated according to actions.
Main Scenario	1. Admin opens Admin → User Management. 2. System displays searchable list of users (username, email, role, status). 3. Admin selects a user. 4. System displays user detail and actions: change role, reset password, deactivate/reactivate. 5. Admin modifies role or status and/or clicks Reset Password. 6. System validates admin permissions. 7. System applies changes to user record. 8. System confirms success.
Alternative Scenarios	N1 – Not allowed to change certain users: at (6) system detects admin cannot demote higher-privilege account → shows "Permission denied". N2 – User not found: at (3) record missing → system shows "User not found".
Non-Functional Constraints	All admin actions must be logged for audit. Reset password must generate secure temporary credentials or link; never show old password.

V. Prototype / Mockup

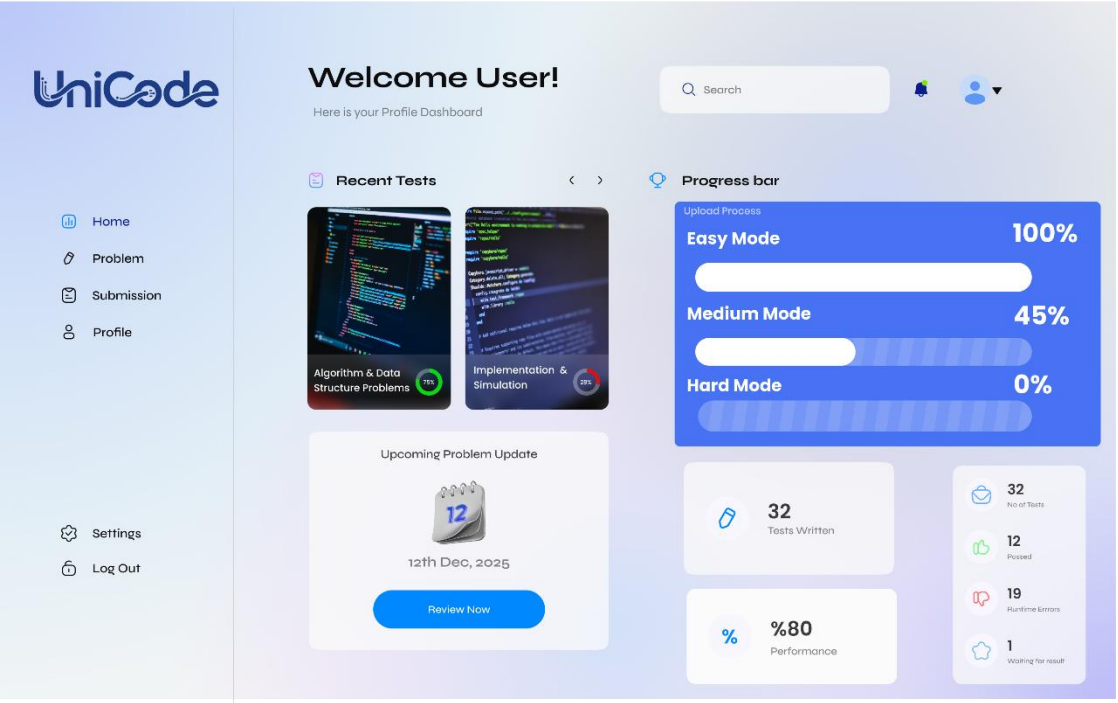
Landing Page



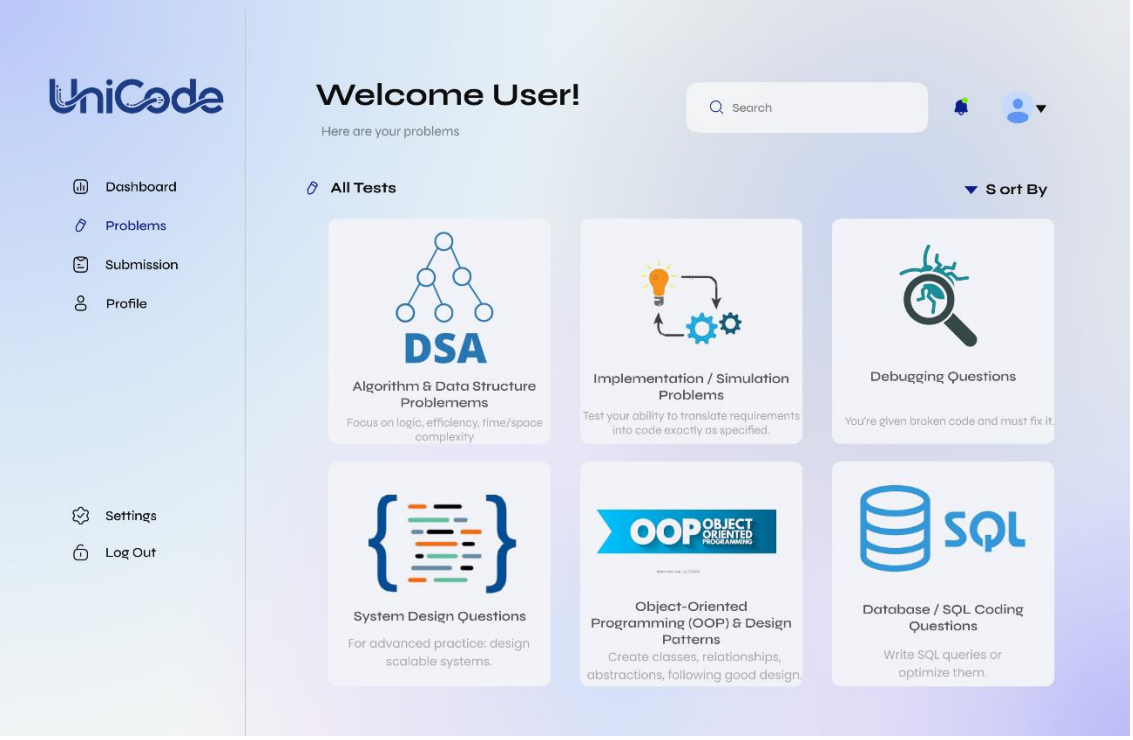
Sign Up Page

Login Page

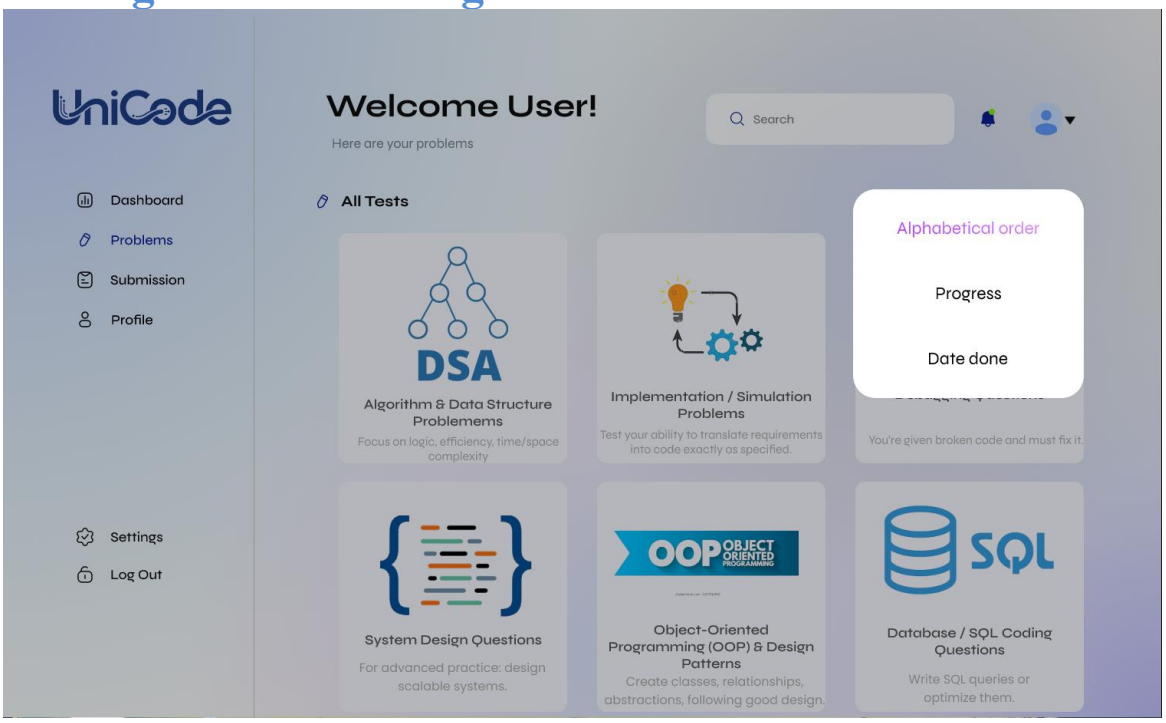
Dashboard



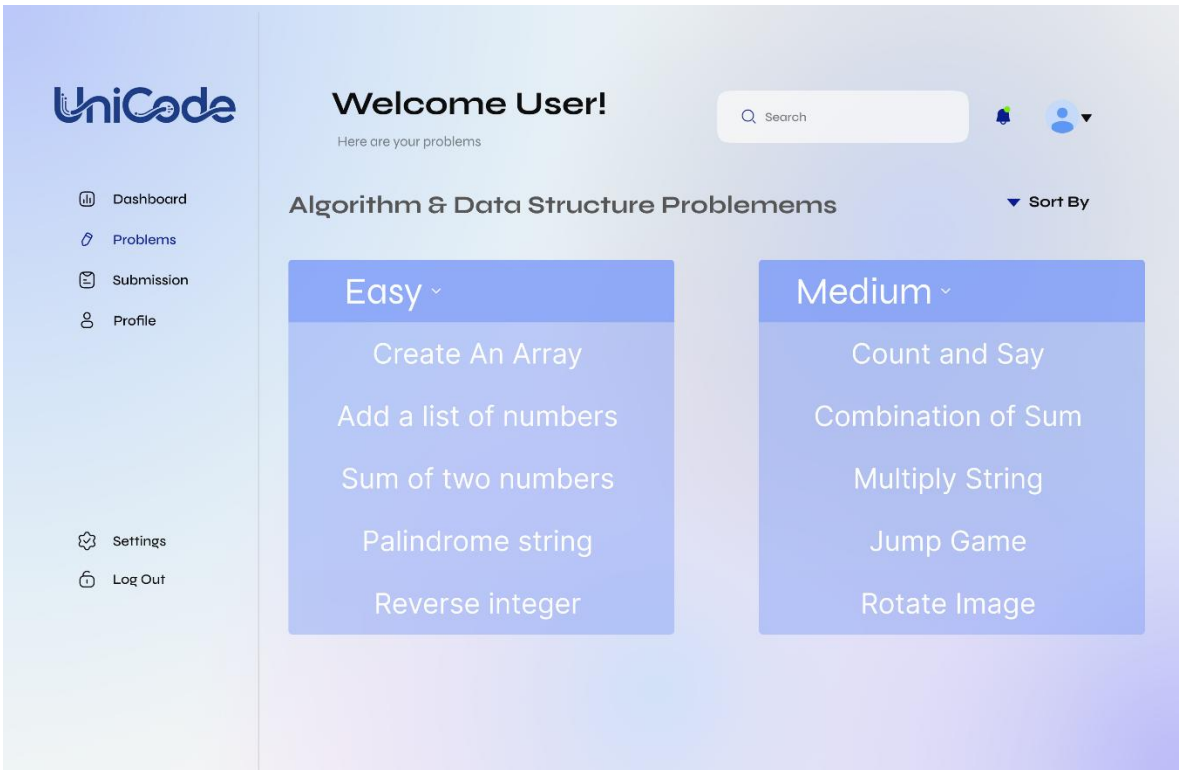
Problems



Sorting in Problem Page:

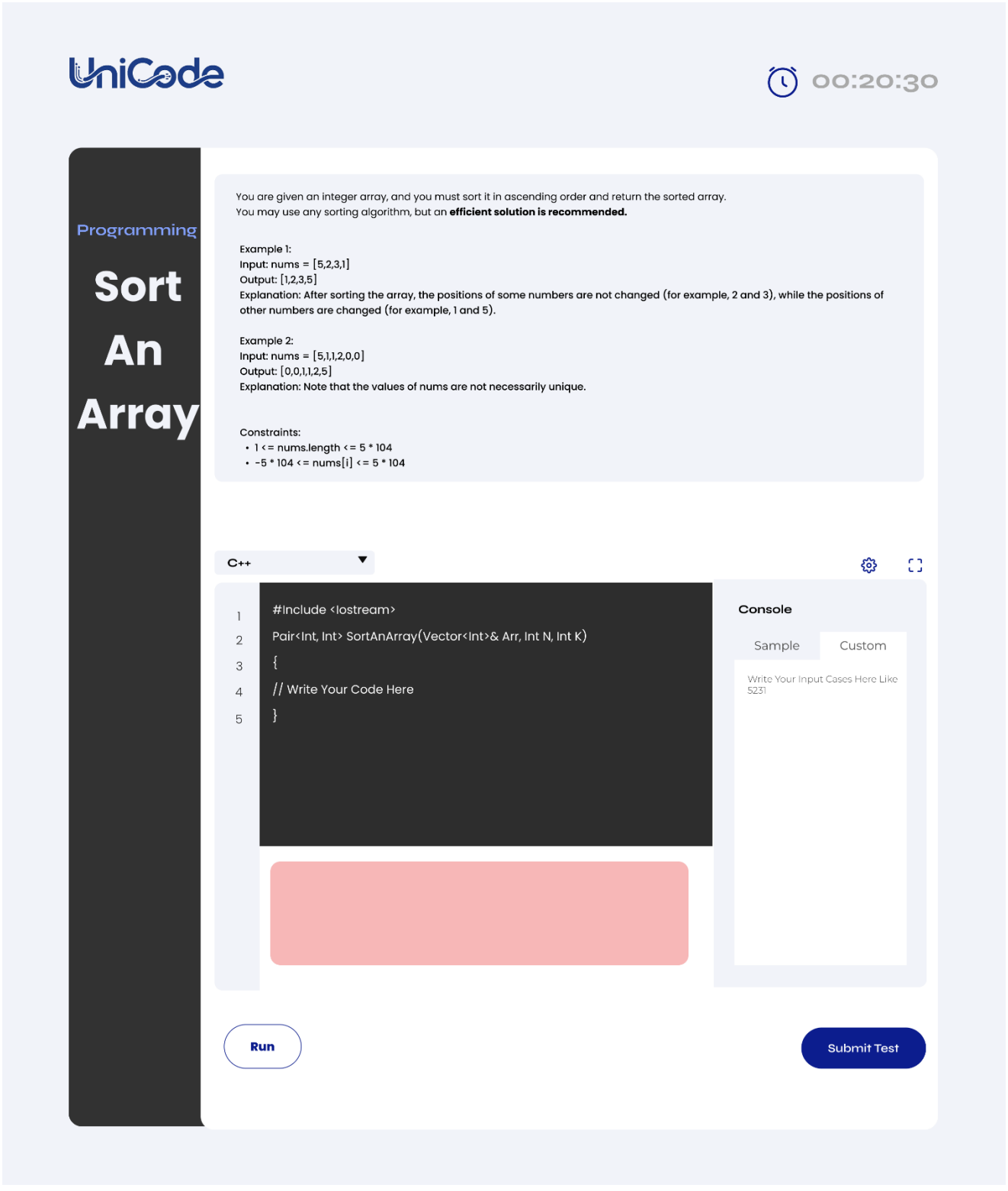


Choosing a Problem:

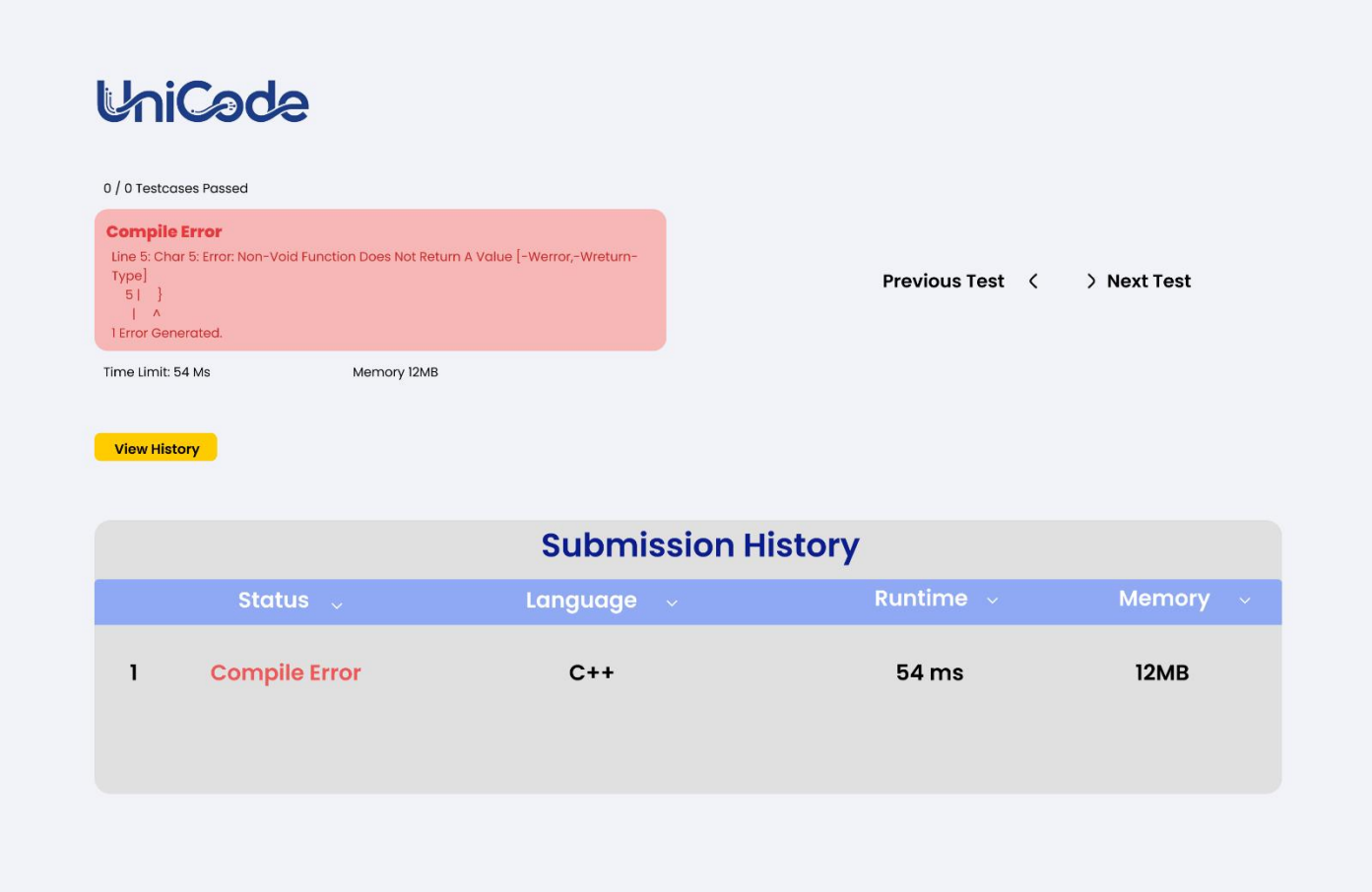


Showing Problem

Coding


UniCode
00:20:30
Programming
Sort An Array
You are given an integer array, and you must sort it in ascending order and return the sorted array.
You may use any sorting algorithm, but an efficient solution is recommended.
Example 1:
Input: nums = [5,2,3,1]
Output: [1,2,3,5]
Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).
Example 2:
Input: nums = [5,1,1,2,0,0]
Output: [0,0,1,1,2,5]
Explanation: Note that the values of nums are not necessarily unique.
Constraints:
• 1 <= nums.length <= 5 * 104
• -5 * 104 <= nums[i] <= 5 * 104
C++
1
2
3
4
5
#Include <Iostream>
Pair<Int, Int> SortAnArray(Vector<Int>& Arr, Int N, Int K)
{
// Write Your Code Here
}
Console
Sample
Custom
Write Your Input Cases Here Like 5231
Run
Submit Test

After submit tests



Link Figma Prototype: https://www.figma.com/proto/hZhpKStZT0mWkgkW4pGpoR/Coding-website?node-id=4471-399&t=oseI7z3QAACRVdgE-1